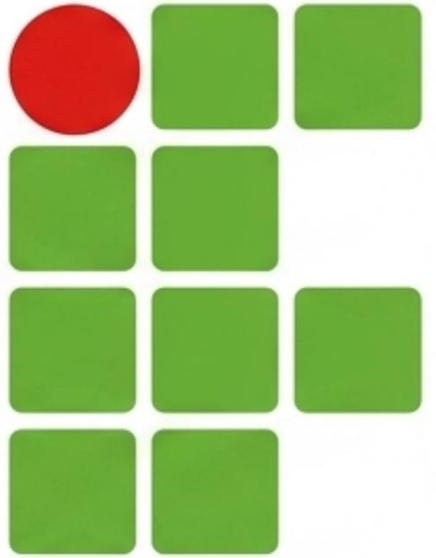


A Arquitetura da Cápsula: O Princípio do Encapsulamento em Java

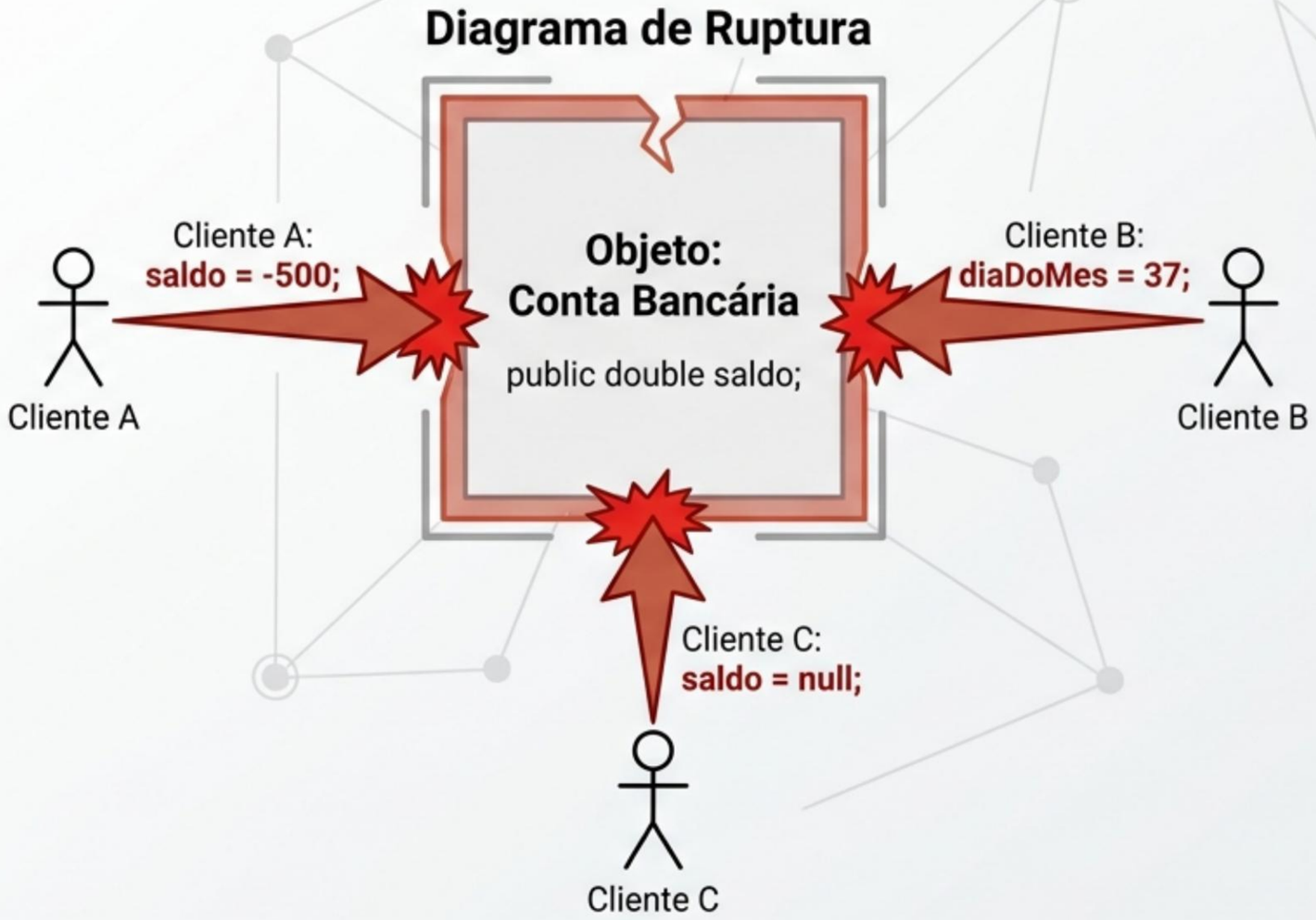
Protegendo dados, isolando falhas e construindo sistemas orientados a objetos robustos.

Prof. Ronaldo Borges

NotebookLM

O Caos dos Dados Expostos

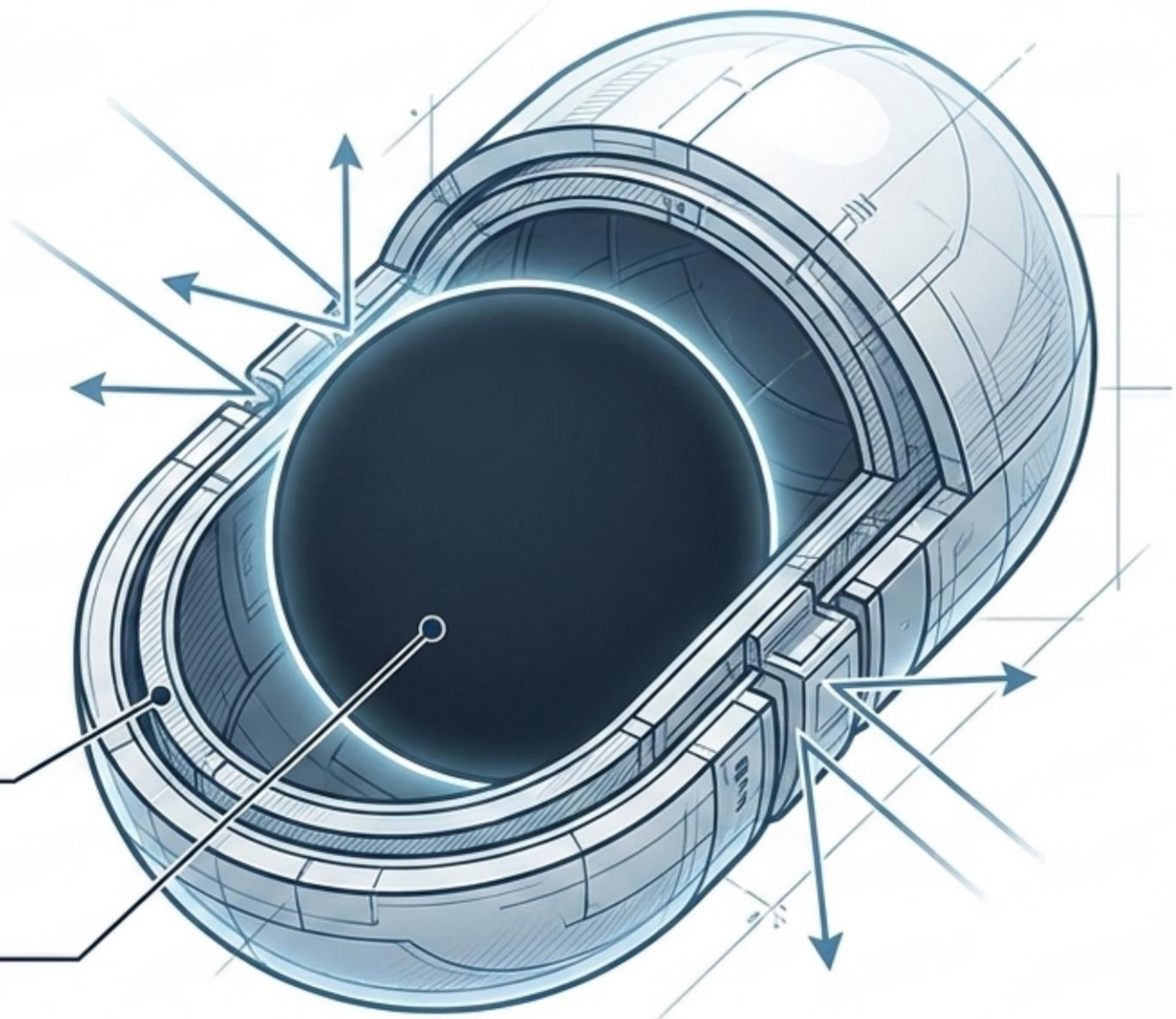
Se as variáveis de instância são públicas, qualquer cliente do sistema pode ver os dados e configurá-los com valores inválidos. O objeto perde completamente o controle sobre o próprio estado.



NotebookLM

A Metáfora da Cápsula (Caixa Preta)

O Encapsulamento é o mecanismo que vincula o código aos dados que ele trata, transformando-os em uma “caixa preta” autônoma. O funcionamento interno é oculto; apenas a **interface** externa é acessível.



O Núcleo (Protegido):
Representa os Dados (Estado)

A Casca (Interface):
Representa o Código (Métodos)

NotebookLM

Os Guardiões do Estado: Modificadores de Acesso

Os modificadores controlam rigorosamente como outras partes do programa interagem com os membros de uma classe. Eles definem a fronteira de confiança do objeto.



public

Acesso total. A interface de serviços.



protected

Acesso para a família (pacote e subclasses).



package (implícito)

Acesso apenas à vizinhança.



private

Isolamento total. Detalhes de implementação.

NotebookLM

A Matriz de Visibilidade em Java

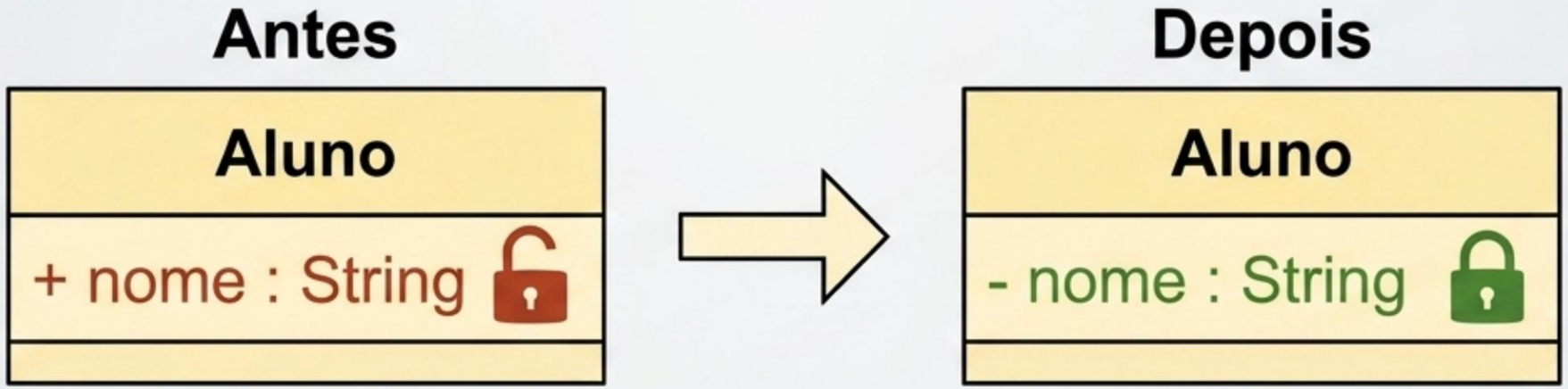
Modificador	Mesma Classe	Mesmo Pacote	Subclasse	Mundo Todo
public	✓	✓	✓	✓
protected	✓	✓	✓	
package (Implícito)	✓	✓		
private	✓			

Atenção: Se nenhum modificador for digitado, o Java assume o acesso “package” por padrão (visível apenas no mesmo pacote).

NotebookLM

Ocultamento de Dados (Data Hiding)

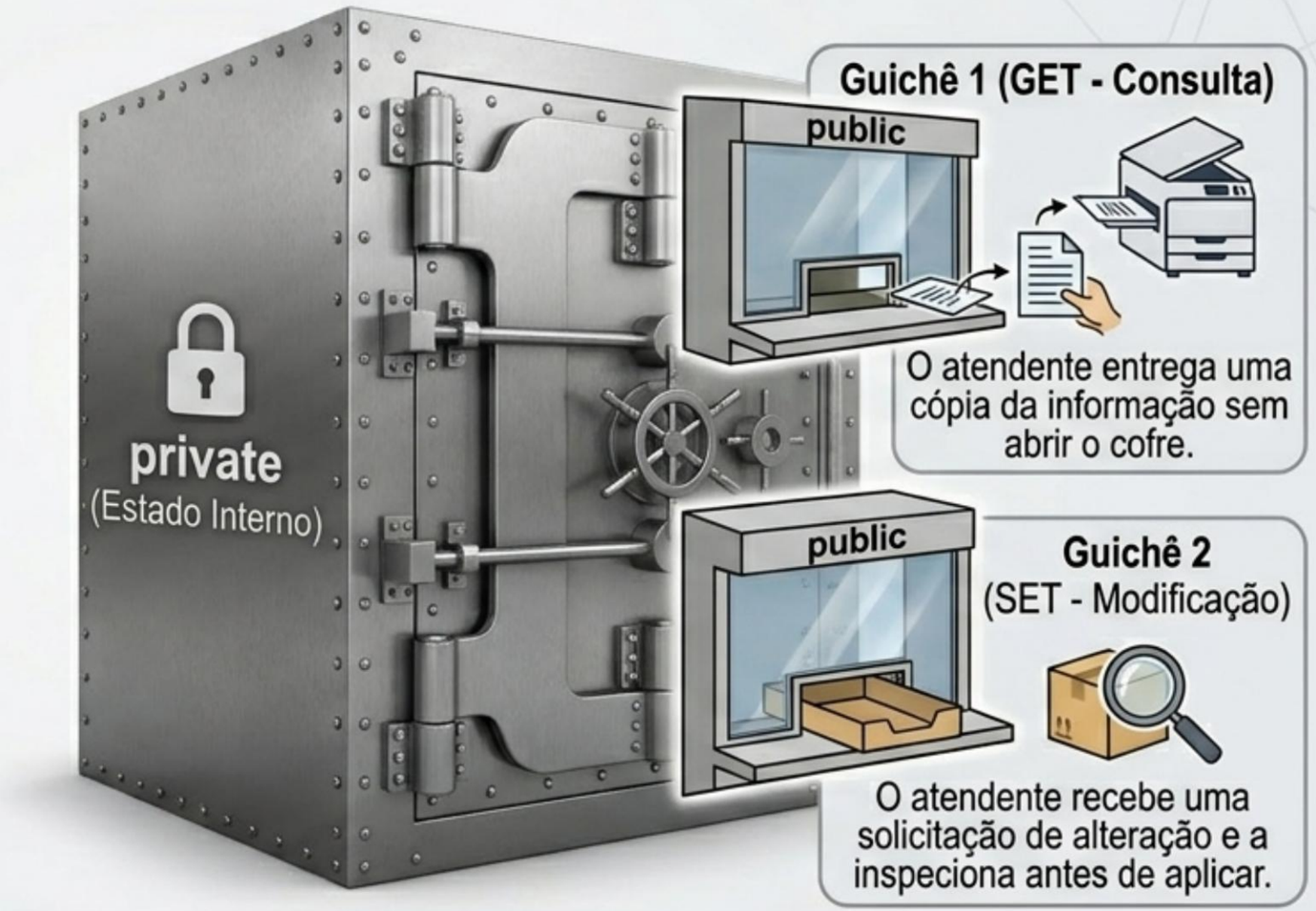
A primeira regra do encapsulamento: Atributos quase sempre devem ser private. Isso protege a variável de interferência direta externa. Os dados estão blindados dentro da classe.



NotebookLM

As Portas de Acesso: Getters e Setters

Métodos de acesso (Getters) e acesso modificação (Setters) criam uma interface public controlada para manipular dados private.



NotebookLM

A Inteligência dos Setters: Validação

Não substitua uma variável pública por um Setter cego. Setters devem examinar cuidadosamente as tentativas de modificação e rejeitar valores inválidos, garantindo a integridade contínua do objeto.

Falso Encapsulamento ⚠

```
this.peso = peso;



setPeso(-10)
```

O objeto aceita o dado corrompido.

Encapsulamento Real ✓

```
if (peso > 0) {
  this.peso = peso;
} else { throw exception;
}

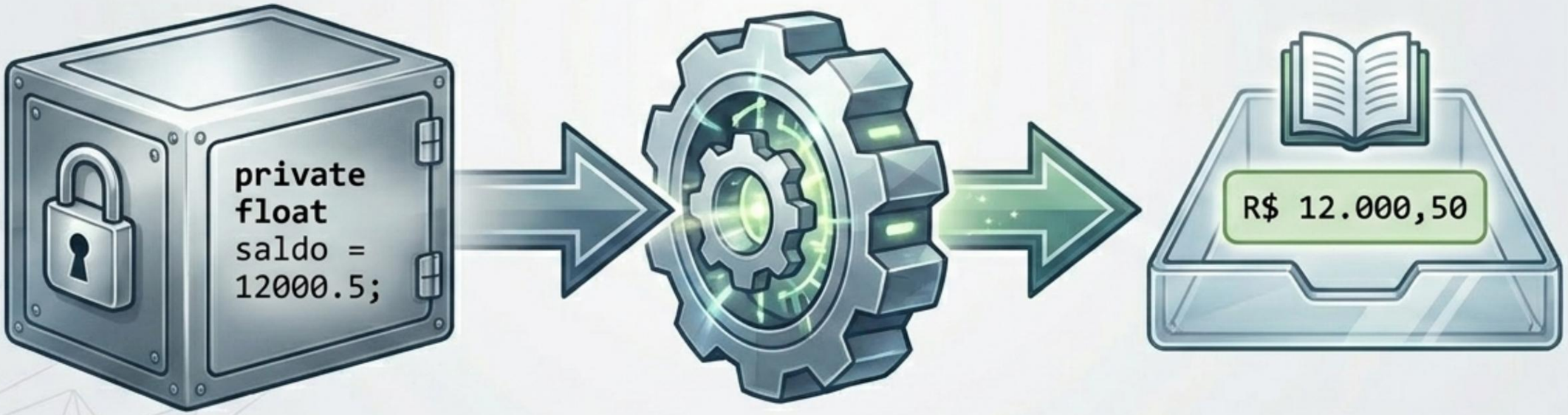
setPeso(-10)
```

Atua como uma barreira de segurança, rejeitando a anomalia.

NotebookLM

O Poder dos Getters: Tradução e Controle

Getters controlam como o cliente visualiza os dados. Isso protege o código do cliente contra mudanças na representação real dos dados internos do objeto.

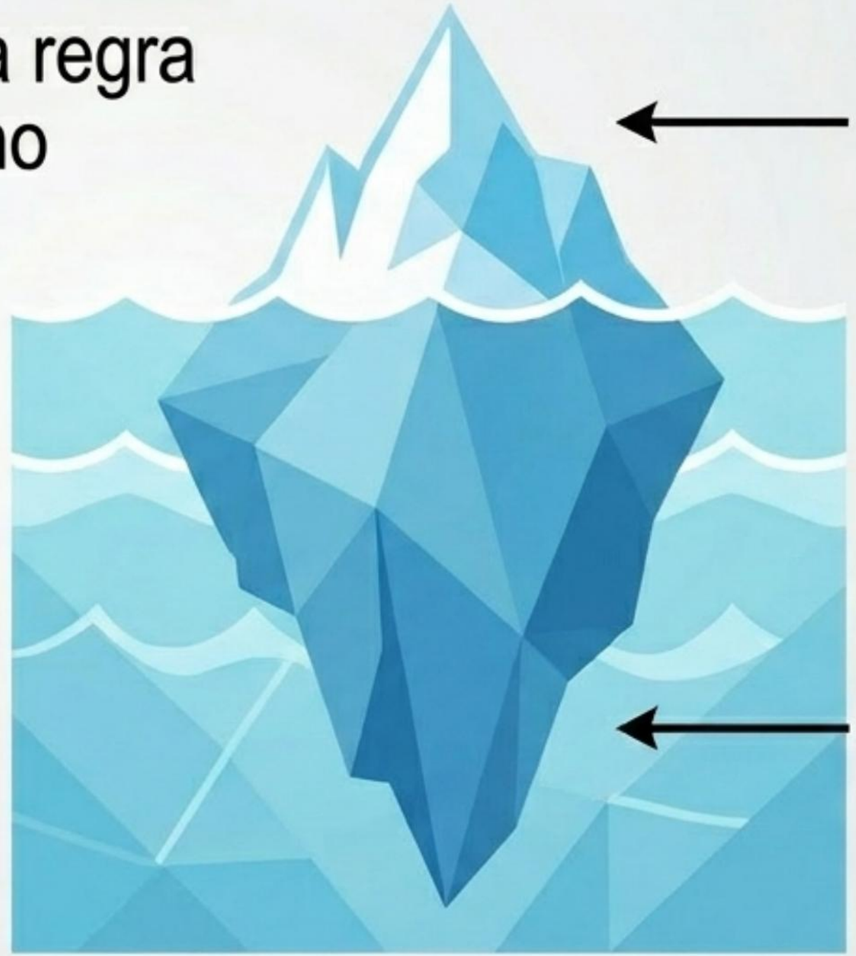


NotebookLM

O Grande Paradigma: Muito Além do ‘Private’

Encapsular não é apenas uma regra sintática para gerar métodos no automático. É um princípio de design focado na autonomia.
Uma aplicação profunda
Uma aplicação profunda do conceito de Abstração: usar

sem precisar entender suas engrenagens internas.



A Ponta (Aspecto Mecânico): Usar `private`, criar métodos **get** e **set**.

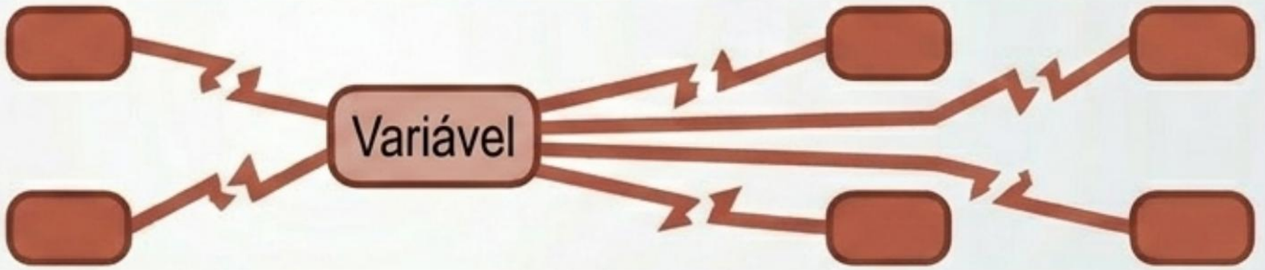
A Base Submersa (O real significado): Autonomia dos objetos, Ocultação de Implementação, Abstração, Redução de Acoplamento.

NotebookLM

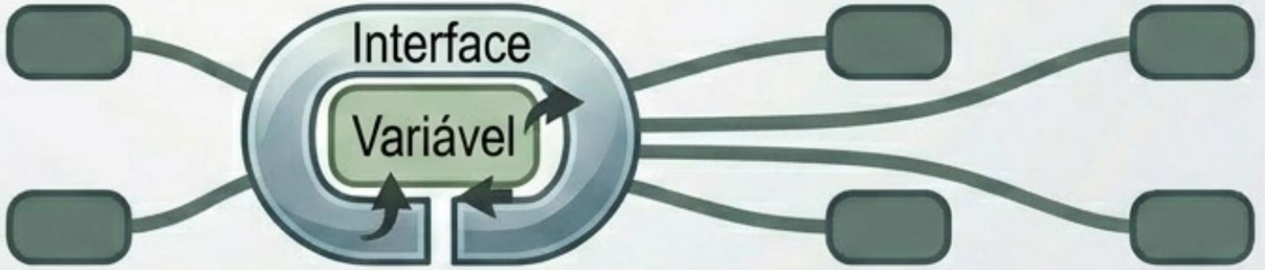
O Impacto no Design de Software

Menor Propagação de Mudanças. Se a implementação de uma operação pode ser substituída sem alterar a interface pública, mantemos as partes do sistema incrivelmente independentes. O código cliente sobrevive intacto.

Arquitetura Frágil
(Sem Encapsulamento)



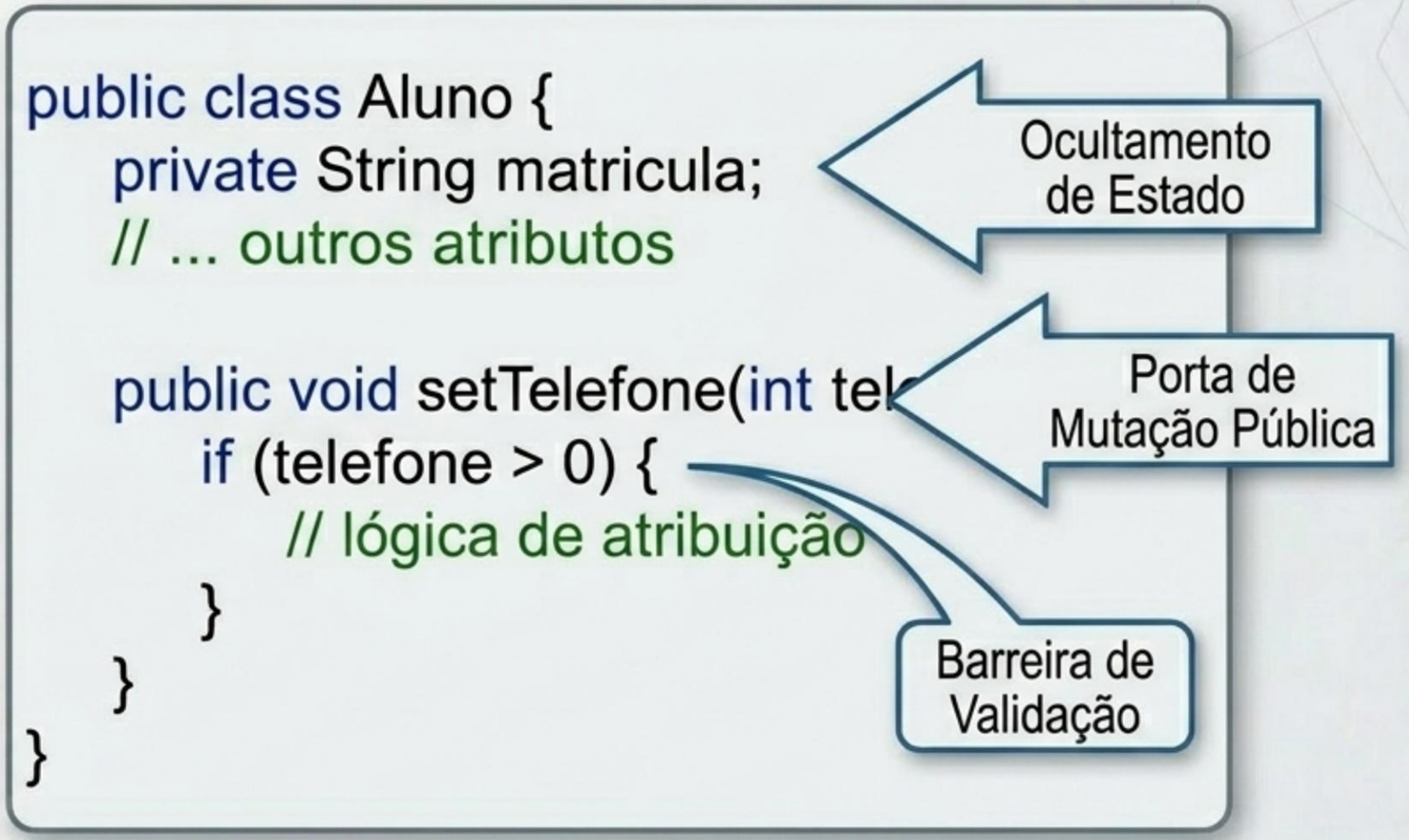
Arquitetura Resiliente
(Com Encapsulamento)



NotebookLM

Anatomia de uma Classe Protegida

A classe é o dispositivo prático do Java que dá suporte à teoria do **encapsulamento**. Dados e operações vitais operando sob o mesmo teto, seguros de interferências externas.



NotebookLM

Referências

BEZERRA, Eduardo. **Princípios de Análise e Projeto de Sistemas com UML**. 3. ed. Rio de Janeiro: Elsevier, 2015.

DEITEL, Paul; DEITEL, Harvey. **Java: Como Programar**. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

IFPI. PPC Tecnologia em Análise e Desenvolvimento de Sistemas. Instituto Federal do Piauí – Campus Floriano, 2023.

SCHILDT, Herbert. Java para iniciantes: crie, compile e execute programas Java. 6. ed. Porto Alegre: Bookman, 2015.

NotebookLM